When a variable's type is incompatible with an operation or call, it is necessary to change it. For example, mathematical operations (such as + or *) work on numbers, and string concatenation (e.g., `"hello" . " world"`) requires string data. This is either done explicitly by the programmer or implicitly by the PHP engine, as mentioned previously. Casting a scalar to another scalar is always possible, with some caveats on what the result might be. Casting compound types to scalar or other compound types isn't usually so straightforward, as we shall see.

PHP will attempt to change scalar data types to suit the call or operation, so the programmer generally does not need to worry about explicitly changing types. This can lead to problems if you are not aware of the rules that PHP will follow. It is also possible to prevent scalar type changes during calls; this is explained in Chapter 3, "Functions."

Converting Data Types

Let's have a quick demonstration of what we've covered so far and play around a bit with data types to see what's really happening.

We're going to use the largest unsigned 16-bit number, *65535*, and the numerical string `"65535."` If you're not sure what "unsigned" means in this context, the detailed explanation can be found early on in Chapter 7, "Numbers"; for now, just accept that it means we don't use up a bit to indicate if the number is positive or negative: unsigned integers are always positive.

PHP will happily switch back and forth between a number and a numerical string as required by the context in which the variable is used. This is known as *type juggling* or *type coercion*; here's a quick demonstration:

```
$str = '65535';
$int = 65535;

var_dump($str);
var_dump($int);

var_dump($int . ' is the largest 16-bit unsigned int.');
// . means 'concatenate these strings'.
var_dump($str + 1); // + means "add these two numbers together".
```